

A Teacher-Anchored Residual Calibration Framework for Drift-Adaptive GKP Decoding

Draft research note for advisor discussion

May 25, 2026

Contents

1	Introduction	2
2	Summary of Contributions	3
3	Brief Review of the GKP Code	4
3.1	Ideal and approximate code states	4
3.2	Syndrome measurement as modular displacement information	4
3.3	Local affine decoding and its limits	4
3.4	Logical failure criterion	5
4	Noise and Drift Setting	5
4.1	Physical noise ingredients	5
4.2	Effective runtime state	5
4.3	Drift scenarios	6
4.4	Connection to affine decoding	6
5	Model Architecture	6
5.1	Fast loop: affine fixed-point decoder	6
5.2	Effective noise state and parameter mapping	7
5.3	Teacher estimators	7
5.4	CNN residual branch	7
5.5	Stage-and-commit runtime contract	8
6	Current Evidence Status and Near-Term Paper Plan	8

Evidence Boundary of This Note

This note is a theory-facing draft for the front matter of a future paper. It does not claim real-board validation, true `.tflite` runtime validation, or paper-grade expanded benchmark completion. The current validated execution boundary remains mock-backed software HIL with `artifact_npz` and `inproc` inference for the recovered paths. Existing frozen-set evidence supports a bounded software revalidation claim, while the post-T55 mechanism story remains partial and should not be written as causal closure.

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction relies on finite-energy oscillator code states and continuous-valued syndrome information to infer displacement errors. In nonstationary hardware settings, however, the effective noise statistics that determine the best syndrome-to-correction map can drift over time, making a fixed decoder suboptimal. This note outlines a two-timescale decoding framework in which a low-latency fast loop applies an FPGA-friendly affine correction, while a slower learned calibration loop updates the affine parameters from recent syndrome statistics. The fast loop executes $\Delta_t = K_t s_t + b_t$, where s_t is the current GKP syndrome and (K_t, b_t) are quantized runtime parameters. The slow loop uses a classical teacher estimator as a stable anchor and trains a lightweight CNN only to predict residual calibration terms, with the current mainline focusing on residual corrections to b_t . This design is motivated by the finite-squeezing structure of physical GKP states, local linear-MMSE approximations to GKP displacement estimation, the modular learned-decoder trend in recent QEC literature, and the need to keep the real-time path deterministic and low latency. The intended contribution is not a generic neural replacement for GKP decoding, but a deployment-constrained, teacher-anchored residual calibration layer for drift-adaptive GKP correction.

1 Introduction

Bosonic codes provide a route to protecting quantum information by encoding a logical qubit into the Hilbert space of an oscillator. Among them, the GKP code is especially attractive because small displacement errors in phase space can be diagnosed by continuous-valued modular syndrome measurements [1]. Physical GKP codewords are necessarily approximate finite-energy states rather than ideal non-normalizable combs, and modern reviews of the code emphasize that this approximation is central to implementation and fault-tolerant scaling [2]. In an idealized setting, a decoder can map each syndrome to a corrective displacement. In realistic devices, however, finite-energy code states, measurement inefficiency, ancilla noise, drift, and calibration error make the syndrome distribution time dependent.

This time dependence creates a practical tension. A high-quality decoder may need to exploit analog soft information and nonstationary noise statistics, as emphasized in recent GKP and concatenated-GKP decoding work [3, 4, 5]. At the same time, real-time quantum error correction imposes strict latency and determinism requirements. Recent surface-code work shows that learned modules can be useful when they are inserted as local, modular, low-latency components rather than as unconstrained replacements for the whole decoding stack [6]. Hardware-conditioned decoders and decoder-prior optimization further suggest that slow calibration signals and hardware statistics can be exploited to improve fast decoding decisions [7, 8].

The central idea of this project is to apply that systems principle to drift-adaptive GKP decoding. The proposed decoder separates the problem into two timescales. The fast loop is deliberately simple: it performs an affine syndrome-informed displacement estimate,

$$\Delta_t = K_t s_t + b_t, \tag{1}$$

using fixed-point arithmetic and a double-buffered parameter bank. The slow loop observes recent syndrome histograms, estimates the current effective noise state, and updates (K_t, b_t) at a much lower rate. Instead of asking the CNN to learn the entire decoder from scratch, the current mainline keeps a classical teacher as the baseline estimator and learns only a residual calibration term.

This framing is intentionally narrower than “a neural GKP decoder.” The method should be understood as a teacher-anchored residual calibration layer for a runtime-constrained affine GKP fast path. That narrower claim is both more faithful to the current implementation and better aligned with the strongest nearby literature.

2 Summary of Contributions

The target paper can be positioned around four contributions.

1. A two-timescale adaptive affine formulation for finite-energy GKP decoding under effective noise drift. We formulate the real-time correction path as an affine estimator $\Delta_t = K_t s_t + b_t$ whose parameters are adapted from recent syndrome statistics. This links the GKP continuous-syndrome picture to a hardware-compatible runtime contract: the expensive inference problem is moved out of the per-shot path, while the fast loop remains a quantized matrix-vector operation. The formulation makes the physical noise ingredients explicit: finite squeezing, displacement noise, measurement noise, bias drift, and covariance rotation are compressed into an effective state that the slow loop can estimate.

2. A teacher-anchored residual calibration strategy. Rather than train a CNN to output all noise parameters or all decoder parameters directly, the mainline uses a classical teacher estimator to produce a stable baseline $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$. The learned model predicts a small residual, currently

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

and the runtime uses

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (3)$$

This design treats the learned module as a calibration layer, not as a full decoder replacement.

3. A deployment-aware runtime architecture. The architecture maps naturally onto a fast/slow implementation. The fast loop uses fixed-point arithmetic, clipping, saturation diagnostics, and a staged parameter bank. The slow loop consumes 32-by-32 syndrome histograms, teacher features, and histogram deltas, then writes new parameters through a stage-and-commit interface. This makes latency, stale-parameter effects, overflow, and commit behavior first-class evaluation objects.

4. A bounded evidence protocol for software-HIL evaluation. The current repository supports a recovered mock-backed software-HIL path and a frozen-set formal software revalidation. These results should be reported with their exact boundary: they support software-level comparison under the frozen scenario/mode set, but they do not yet constitute true `.tflite` runtime validation, real-board validation, or an expanded paper-grade benchmark. A future full paper should add stronger comparator lanes, unseen drift families, runtime deployment checks, and mechanism-hedged ablation tables before making broader claims.

3 Brief Review of the GKP Code

3.1 Ideal and approximate code states

The GKP code encodes a qubit into an oscillator by arranging logical code states as periodic combs in phase space [1]. In the convention used by this project, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Physical GKP states must be approximate finite-energy states, so the comb peaks are broadened and enveloped. This finite-energy structure is not a detail: it sets an intrinsic noise floor and affects the syndrome statistics seen by the decoder.

Following the standard finite-energy picture summarized by Grimsmo and Puri [2], an approximate GKP codeword can be viewed as an ideal comb acted on by an energy-damping envelope. A common mathematical model is

$$|\widetilde{\mu}_L\rangle \propto e^{-\Delta^2 \hat{a}^\dagger \hat{a}} |\mu_L\rangle, \quad (6)$$

where Δ controls the envelope and peak width, and the ideal code is recovered only in the unphysical limit $\Delta \rightarrow 0$. Equivalently, finite squeezing turns exact lattice spikes into Gaussian peaks under a global envelope. This has two consequences for the present project. First, the syndrome is intrinsically analog and noisy even before additional hardware noise is applied. Second, the decoder should treat the observed histogram as a finite-energy, finite-resolution statistical object rather than as an exact ideal-code syndrome distribution.

3.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (7)$$

An ideal GKP syndrome measures this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (8)$$

Real measurements add finite-squeezing, inefficiency, shot, and ancilla contributions. The measured syndrome is better viewed as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}}. \quad (9)$$

Thus the decoder does not observe the absolute displacement; it observes a noisy modular representative of that displacement.

3.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Within a local branch and under a Gaussian approximation, however, the error e and syndrome s can be treated as jointly Gaussian:

$$\begin{bmatrix} e \\ s \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_e \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{ee} & \Sigma_{es} \\ \Sigma_{se} & \Sigma_{ss} \end{bmatrix} \right). \quad (10)$$

The linear-MMSE estimate is then

$$\hat{e} = \mu_e + \Sigma_{es}\Sigma_{ss}^{-1}(s - \mu_s) = Ks + b, \quad (11)$$

where

$$K = \Sigma_{es}\Sigma_{ss}^{-1}, \quad b = \mu_e - K\mu_s. \quad (12)$$

This explains why the affine form is a reasonable fast-path approximation. It also explains its limitation: near lattice decision boundaries, the posterior may be multimodal, and a single global affine estimator may average over branches. The proposed architecture therefore does not claim Bayes-optimal GKP decoding; it seeks a low-latency adaptive approximation.

3.4 Logical failure criterion

After applying a correction, the residual displacement is wrapped back into the GKP fundamental cell. A logical error occurs when the residual crosses the logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (13)$$

and similarly for the p -quadrature. This is why closed-loop logical error rate is a stronger evaluation target than offline parameter-regression error.

4 Noise and Drift Setting

4.1 Physical noise ingredients

The theoretical noise model used by this project separates physical ingredients from the low-dimensional effective state exposed to the runtime decoder. The physical ingredients are:

1. finite GKP squeezing, represented as broadened comb peaks and an energy envelope in the approximate code state;
2. additive displacement noise in the q and p quadratures;
3. measurement noise, including finite readout precision, inefficiency, shot noise, and auxiliary-state imperfections;
4. coherent or slowly varying displacement bias, represented by a nonzero mean (μ_q, μ_p) ;
5. anisotropic or rotated covariance, represented by different principal widths and an angle ϑ .

Photon loss, thermal excitation, dephasing, and ancilla faults can all be projected into this effective displacement-and-measurement picture over the timescale of the decoder update. This projection is not meant to erase the underlying physics; it is the interface that lets a low-latency affine fast path respond to hardware drift without carrying a full device-level simulator in the real-time loop.

4.2 Effective runtime state

The slow loop summarizes the current noise condition as

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (14)$$

or, in the anisotropic version, as

$$\theta_t^{\text{noise}} = (\sigma_{q,t}, \sigma_{p,t}, \mu_{q,t}, \mu_{p,t}, \vartheta_t). \quad (15)$$

Here σ controls the effective displacement scale, μ_q and μ_p encode syndrome-center bias, and ϑ rotates the covariance axes. The teacher and CNN do not need to identify every microscopic noise source separately; they need to estimate the effective statistics that determine the affine correction parameters.

4.3 Drift scenarios

The project treats drift as slow time variation in θ_t^{noise} . Four canonical scenarios define the current mainline drift vocabulary:

1. **static bias**: a stationary nonzero displacement mean or fixed covariance orientation;
2. **linear ramp**: gradual monotonic change in scale, bias, or orientation;
3. **step change**: an abrupt calibration jump, such as a sudden change in σ_t or ϑ_t ;
4. **periodic drift**: oscillatory variation caused by repeatable calibration or environmental cycles.

These scenarios are sufficient to define a clean adaptive-decoding problem: a fixed affine decoder is calibrated for one distribution, while the actual syndrome distribution moves. A slow-loop estimator should track the changing histogram and stage new (K, b) parameters before the fast path becomes badly mismatched.

4.4 Connection to affine decoding

Under a local Gaussian approximation, the effective state determines the affine map. Increased displacement scale changes the optimal gain K ; mean bias changes the offset b ; covariance rotation introduces cross-quadrature coupling; measurement noise reduces the useful information in the syndrome and therefore shrinks the gain. This is why the runtime interface is written in terms of (K, b) rather than a free-form neural correction. The neural branch can improve calibration only insofar as it produces better runtime parameters for the current effective noise-and-drift state.

5 Model Architecture

5.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome s_t and reads the active parameter bank. Its runtime computation is

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\text{max}}, s_{\text{max}}), \quad (16)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (17)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\text{max}}, \Delta_{\text{max}})), \quad (18)$$

where $Q(\cdot)$ denotes fixed-point quantization. The project convention is an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also accumulates syndrome histograms and records diagnostics such as histogram-input saturation, correction saturation, and aggressive-parameter events.

5.2 Effective noise state and parameter mapping

The slow loop models the current noise condition by a low-dimensional effective state

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t). \quad (19)$$

Given an estimate of this state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (20)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (21)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (22)$$

followed by eigenvalue clipping and optional gain scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (23)$$

Finally, exponential smoothing produces the staged runtime parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (24)$$

5.3 Teacher estimators

The teacher family provides stable, interpretable estimates of θ_t^{noise} from recent syndrome history. The simplest teacher computes first- and second-order moments from the histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add a state-space prior over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (25)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (26)$$

The teacher is not a rhetorical device; it is the stability anchor that prevents the learned module from becoming an unconstrained decoder.

5.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (27)$$

Teacher-side features are represented as

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots), \quad (28)$$

although the current gated branch narrows this to a small set of teacher- b and teacher- Δb scalars. The learned branch predicts

$$\hat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (29)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \widehat{\delta b}_t, -b_{\max}, b_{\max}). \quad (30)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (31)$$

The residual formulation is important: it makes the CNN responsible for a small deployment-relevant correction rather than for the full GKP decoding problem.

5.5 Stage-and-commit runtime contract

The slow loop does not directly mutate the active fast-loop parameters. It stages a candidate (K, b) into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (32)$$

This contract is central to the deployment story. It allows the paper to evaluate not only logical error rate, but also update latency, stale-parameter effects, commit success, rollback/fallback behavior, and fixed-point stability.

6 Current Evidence Status and Near-Term Paper Plan

The current safe result story is bounded. The repository supports a recovered mock-backed software-HIL path and a frozen four-scenario, five-mode software revalidation in which `hybrid-residual_b` ranks first within that frozen set. The claim must remain tied to that frozen set and to the software HIL boundary. It should not be generalized to true `.tflite`, real-board HIL, or a broad state-of-the-art comparison.

The mechanism story also requires care. Earlier trace analysis suggested that committed- b amplitude and residual clipping were involved in seed-dependent Gated-v5 behavior. The T55 lower-clip intervention, however, was mixed and mostly harmful across the locked six-seed pack. Therefore the correct paper wording is not that “large committed b is the root cause.” A safer statement is that the learned residual branch can enter seed- and scenario- dependent high-amplitude regimes whose effect is not captured by a simple monotonic amplitude explanation.

Before a full submission, the most important additions are:

1. a benchmark expansion lane that keeps the frozen-set anchor but adds unseen drift families and stronger calibration/statistical baselines;
2. a mechanism table that distinguishes retained, weakened, retired, and still-open hypotheses after T55;
3. true runtime-boundary evidence for `.tflite` if the dependency environment becomes available;
4. fixed-point and latency evidence that treats deployment constraints as measured quantities rather than prose assumptions.

References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman-Kitaev-Preskill Code,” *PRX Quantum*, vol. 2, p. 020101, 2021. doi: 10.1103/PRXQuantum.2.020101.
- [3] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low overhead fault-tolerant quantum error correction with the surface-GKP code,” *npj Quantum Information*, 2022.
- [4] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault Tolerant Decoding of QLDPC-GKP Codes with Circuit Level Soft Information,” preprint, 2025.
- [5] M.-A. Roy, T. Pousset, and B. Royer, “Decoding Multimode Gottesman-Kitaev-Preskill Codes with Noisy Auxiliary States,” preprint, 2025.
- [6] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.
- [7] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-Conditioned FiLM Decoders for Low-Latency Decoding of Quantum Error Correction Evaluated on IBM Repetition-Code Experiments,” preprint, 2026.
- [8] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Nature*, 2024.
- [9] X. Yang, X. Sun, Z. Wu, J. Zhang, J. Jiang, X. Linpeng, Y. Zhou, J. Chu, J. Niu, Y. Zhong, S. Liu, and D. Yu, “Real-time Surface-Code Error Correction Using an FPGA-based Neural-Network Decoder,” preprint, 2026.